

Analysis of Frontend API and frontend to backend connectivity using angular

→ The components does not have test files , may be when AI creates components it might skips the test files

Models in Banking App

LoginModel:

```
export class LoginModel {  
    constructor(public username: string = "", public password: string = "")  
    {}  
}
```

- Can have default values (username = "", password = "")
- Can have constructors - initialization logic
- Can have methods - business logic, validation
- Can be instantiated with new LoginModel()
- Can implement interfaces
- Can be extended via inheritance
- Exists at runtime - type information is preserved

Models in BusBooking app:

User and AuthResponse (Interfaces)

```
export interface User {  
    id: string;  
    fullName: string;  
    email: string;  
    role: UserRole;  
    isActive: boolean;  
}  
  
export interface AuthResponse {  
    token: string;  
    refreshToken: string;  
    user: User;  
}
```

- Cannot have default values - all properties required (unless using ? optional)
- Cannot have constructors
- Cannot have methods - only property definitions
- Cannot be instantiated with new
- Cannot be extended (though they can extend other interfaces)

- ➔ **Only exist at compile-time** - no runtime representation (they get erased)
- ➔ **Lighter weight** - better for pure data structures
- ➔ **Multiple inheritance** - can extend multiple interfaces

Usage ex:

// Must provide all required properties

```
const user: User = {
  id: "123",
  fullName: "Padmavathi SJ",
  email: "padmavathi@example.com",
  role: UserRole.Admin,
  isActive: true
};
```

// Cannot do this:

```
// const user = new User(); // Error! Interface cannot be instantiated
```

➔ Use class when:

// 1. Need default values

```
class LoginModel {
  constructor(
    public username: string = "",
    public password: string = ""
  ) {}
}
```

// 2. Need validation/utility methods

```
class UserModel {
  constructor(public user: User) {}

  getDisplayName(): string {
    return `${this.user.fullName} (${this.user.email})`;
  }

  isAdmin(): boolean {
    return this.user.role === UserRole.Admin;
  }
}
```

// 3. Need to create instances with logic

```
const loginForm = new LoginModel();
```

Use Interface when:

// 1. API response/request shapes

```
interface ApiResponse {
  status: number;
```

```
    data: any;
    message: string;
}
```

// 2. Props for components

```
interface ComponentProps {
  title: string;
  onClick: () => void;
}
```

// 3. Configuration objects

```
interface Config {
  apiUrl: string;
  timeout: number;
  retries: number;
}
```

// 4. Type checking for plain objects

```
const userData: User = {
  id: "1",
  fullName: "John Doe",
  email: "john@example.com",
  role: UserRole.Customer,
  isActive: true
};
```

AdminService in BusBookingApp:

→ Injectable decorator to make this service available for dependency injection
import { Injectable } from '@angular/core';

→ HttpClient for making API calls and HttpParams for query parameters

import { HttpClient, HttpParams } from '@angular/common/http';

→ Observable for handling asynchronous data streams

import { Observable } from 'rxjs';

→ environment configuration (contains api urls, environment settings)

import { environment } from '../environments/environment';

→ interface defining the structure of operator request/application data

export interface **OperatorRequest** {

id: string;

fullName: string;

```
email: string;
phone: string;
companyName: string;
licenseNumber: string;
address: string;
status: number;
createdAt: string;
}
```

→ interface defining the structure of Bus station data

```
export interface Station {
  id: string;
  name: string;
  city: string;
  state: string;
  code: string;
  isActive: boolean;
}
```

→ interface defining the structure of Route data

```
export interface Route {
  id: string;
  name: string;
  source: string;
  destination: string;
  distanceKm: number;
  isActive: boolean;
}
```

→ interface defining the structure of Bus data

```
export interface Bus {  
  id: string;  
  operatorId: string;  
  operatorName: string;  
  routeId: string;  
  routeName: string;  
  busNumber: string;  
  busName: string;  
  busType: string;  
  totalSeats: number;  
  femaleSeats: number;  
  maleSeats: number;  
  status: number;  
}
```

→ @injectable decorator makes this service available for dependency injection

→ providedIn: 'root' means angular will create a single, shared instance of this service

```
@Injectable({  
  providedIn: 'root'  
})
```

```
export class AdminService {  
  private apiUrl = `${environment.apiUrl}/admin`;
```

→ constructor with dependency injection – HttpClient is automatically provided by angular

```
constructor(private http: HttpClient) {}
```

→ @param status – optional status number to filter operators

→ returns observable array of operatorRequest objects

```
getOperators(status?: number): Observable<OperatorRequest[]> {
```

→ create new HttpParams object for query parameters

```
    let params = new HttpParams();  
    if (status !== undefined && status !== null) {  
        params = params.set('status', status.toString());  
    }  
    return this.http.get<OperatorRequest[]>(`${this.apiUrl}/operators`, { params });  
}
```

```
getPendingOperators(): Observable<OperatorRequest[]> {  
    return this.http.get<OperatorRequest[]>(`${this.apiUrl}/operator-requests`);  
}
```

```
approveOperator(id: string): Observable<any> {  
    return this.http.post(`${this.apiUrl}/approve-operator/${id}`, {});  
}
```

```
rejectOperator(id: string, reason: string): Observable<any> {  
    return this.http.post(`${this.apiUrl}/reject-operator/${id}`, JSON.stringify(reason), {  
        headers: { 'Content-Type': 'application/json' }  
    });  
}
```

```
enableOperator(id: string): Observable<any> {  
    return this.http.post(`${this.apiUrl}/enable-operator/${id}`, {});  
}
```

```
disableOperator(id: string): Observable<any> {  
    return this.http.post(`${this.apiUrl}/disable-operator/${id}`, {});  
}
```

```
}
```

→ Stations

```
getStations(): Observable<Station[]> {  
    return this.http.get<Station[]>(`${this.apiUrl}/stations`);  
}
```

```
createStation(station: any): Observable<any> {  
    return this.http.post(`${this.apiUrl}/stations`, station);  
}
```

→ Routes

```
getRoutes(): Observable<Route[]> {  
    return this.http.get<Route[]>(`${this.apiUrl}/routes`);  
}
```

```
createRoute(route: any): Observable<any> {  
    return this.http.post(`${this.apiUrl}/routes`, route);  
}
```

→ Buses

```
getPendingBuses(): Observable<Bus[]> {  
    return this.http.get<Bus[]>(`${this.apiUrl}/pending-buses`);  
}
```

```
getAllBuses(source?: string, destination?: string): Observable<Bus[]> {  
    let params = new HttpParams();  
    if (source) params = params.set('source', source);  
    if (destination) params = params.set('destination', destination);
```

```
return this.http.get<Bus[]>(`${this.apiUrl}/all-buses`, { params });  
}
```

```
getBusRequests(): Observable<Bus[]> {  
    return this.http.get<Bus[]>(`${this.apiUrl}/fleet/requests`);  
}
```

```
getActiveBuses(): Observable<Bus[]> {  
    return this.http.get<Bus[]>(`${this.apiUrl}/fleet/active`);  
}
```

```
getAllHistoryBuses(): Observable<Bus[]> {  
    return this.http.get<Bus[]>(`${this.apiUrl}/fleet/history`);  
}
```

```
approveBus(id: string): Observable<any> {  
    return this.http.post(`${this.apiUrl}/approve-bus/${id}`, {});  
}
```

```
rejectBus(id: string, reason: string): Observable<any> {  
    return this.http.post(`${this.apiUrl}/reject-bus/${id}`, JSON.stringify(reason), {  
        headers: { 'Content-Type': 'application/json' }  
    });  
}
```

```
addBus(bus: any): Observable<any> {  
    return this.http.post(`${this.apiUrl}/add-bus`, bus);  
}  
}
```

/backrnd/BusBooking.API/program.cs:

→ CORS

```
var allowedOrigins = builder.Configuration
    .GetSection("Cors:AllowedOrigins").Get<string[]>(!);
```

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngular", policy =>
        policy.WithOrigins(allowedOrigins)
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials());
});
```

appSettings.json:

```
"AllowedHosts": "*",
"Cors": {
  "AllowedOrigins": [
    "http://localhost:4200"
  ]
}
```

→ I have handled cors to allow the frontend origin , by explicitly mentioning the domain and port.

User-Login.ts:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router, RouterModule, ActivatedRoute } from '@angular/router';
import { AuthService } from '../core/services/auth.service';
```

```
import { MatCardModule } from '@angular/material/card';
import { MatInputModule } from '@angular/material/input';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { CommonModule } from '@angular/common';
```

```
@Component({
  selector: 'app-user-login',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule, RouterModule, MatCardModule, MatInputModule,
MatButtonModule, MatIconModule],
  templateUrl: './user-login.html',
  styleUrls: ['./login/login.scss']
})
```

```
export class UserLogin implements OnInit {
```

```
  loginForm: FormGroup;
```

```
  hidePassword = true;
```

```
  isLoading = false;
```

```
  errorMessage = "";
```

```
  returnUrl = '/user';
```

```
  constructor(
```

```
    private fb: FormBuilder,
```

```
    private authService: AuthService,
```

```
    private router: Router,
```

```
    private route: ActivatedRoute
```

```
  ) {
```

```
    this.loginForm = this.fb.group({
```

```
      email: ["", [Validators.required, Validators.email]],
```

```

    password: ['', [Validators.required, Validators.minLength(6)]]
  });
}

ngOnInit() {
  // Get return URL from route parameters or default to '/user'
  this.returnUrl = this.route.snapshot.queryParams['returnUrl'] || '/user';
}

onSubmit() {
  if (this.loginForm.invalid) return;

  this.isLoading = true;
  this.errorMessage = "";

  this.authService.userLogin(this.loginForm.value).subscribe({
    next: () => {
      // Navigate to return URL or default to /user
      this.router.navigateByUrl(this.returnUrl);
    },
    error: (err) => {
      this.errorMessage = err.error.message || 'Login failed';
      this.isLoading = false;
    }
  });
}
}

```

Component Decorator:

```
@Component({
  selector: 'app-user-login',
  standalone: true, // Standalone component (doesn't need NgModule)
  imports: [CommonModule, ReactiveFormsModule, RouterModule, MatCardModule, MatInputModule,
    MatButtonModule, MatIconModule],
  templateUrl: './user-login.html',
  styleUrls: ['./login/login.scss']
})
```

→ Standalone component → modern angular approach, this does not need NgModule

Constructor and Form Initialization

```
constructor(private fb: FormBuilder, private authService: AuthService,
  private router: Router, private route: ActivatedRoute) {
  this.loginForm = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]]
  });
}
```

- ➔ **FormBuilder:** Creates form controls with validation
- ➔ **Email validation:** Required + valid email format
- ➔ **Password validation:** Required + minimum 6 characters

ngOnInit Lifecycle Hook

```
ngOnInit() {
  this.returnUrl = this.route.snapshot.queryParams['returnUrl'] || '/user';
}
```

- Gets return URL from query parameters (e.g., /login?returnUrl=/dashboard)
- Falls back to '/user' if no return URL specified

OnSubmit method:

```
onSubmit() {  
  
  if (this.loginForm.invalid) return; // Early exit if form invalid  
  
  this.isLoading = true; // Show loading spinner  
  
  this.errorMessage = ""; // Clear previous errors  
  
  this.authService.userLogin(this.loginForm.value).subscribe({  
  
    next: () => {  
  
      this.router.navigateByUrl(this returnUrl); // Redirect on success  
  
    },  
  
    error: (err) => {  
  
      this.errorMessage = err.error.message || 'Login failed';  
  
      this.isLoading = false; // Hide loading on error  
  
    }  
  
  });  
}
```

→ here used the Observable logic

- ➔ Next
- ➔ Error
- ➔ Complete

I have used FormGroup -> to group multiple controls together, like a container that manages a collection of FormControl instances.

// FormGroup is like a JavaScript object that holds multiple form controls

```
import { FormGroup, FormControl } from '@angular/forms';
```

```
// A FormGroup contains multiple FormControls
```

```
const formGroup = new FormGroup({  
  email: new FormControl(""), // FormControl 1  
  password: new FormControl("") // FormControl 2  
});
```

```
// Access individual controls
```

```
formGroup.get('email'); // Returns the email FormControl
```

```
formGroup.get('password'); // Returns the password FormControl
```

```
// Get all values
```

```
console.log(formGroup.value); // { email: "", password: "" }
```

ex:

```
loginForm = {  
  // FormGroup properties  
  value: { email: "", password: "" },  
  valid: false,  
  invalid: true,  
  touched: false,  
  untouched: true,  
  dirty: false,  
  pristine: true,
```

```
// FormControls inside

controls: {

  email: {

    value: "",

    valid: false,

    errors: { required: true },

    touched: false

  },

  password: {

    value: "",

    valid: false,

    errors: { required: true, minlength: true },

    touched: false

  }

}

}
```

Benefits:

1. **Organized Structure:** Groups related controls together
2. **Bulk Operations:** Validate, enable, disable all controls at once
3. **Cross-field Validation:** Validate relationships between fields
4. **Simplified Template:** Single [formGroup] binding
5. **Easy Reset:** Reset entire form with one method
6. **Value Aggregation:** Get all values as single object
7. **Status Tracking:** Track overall form validity, dirtiness, etc.